

Diagnosing Local LLM Agent Tool Use Under Clean-Room Conditions

A repeated controlled and runtime-native benchmark of six local models

Emmanuel Belo

Local Agent Bench

Dataset: clean-room-2026-07

Draft date: 2026-07-09

Abstract. Local large language models are increasingly used as agent backends, but local tool-use failures are often reported as undifferentiated model failures. This paper presents Local Agent Bench, a diagnostic benchmark for local LLM agent tool use, and reports a clean-room repeated evaluation of six 7B-9B class models served through Ollama. The study separates three layers: a primary benchmark-owned ReAct protocol through raw Ollama, a secondary controlled ReAct comparison through OpenClaw, Hermes, and Pi CLI runtimes, and a secondary platform-native diagnostic phase in which those runtimes own tool use.

The clean-room run used Docker Compose on WSL2 with an NVIDIA RTX PRO 1000 8 GB GPU, Ollama 0.31.1, and image digest `ollama/ollama@sha256:f1a705f2bd113fb8d15f85f7c217f0dc5f6bebda6b0cc42b82c3ad165ffcb9dc`. Runtime phases used a separate clean overlay with OpenClaw 2026.6.11, Pi 0.80.3, Hermes pinned to `f64e4f4f5768c18a53f44890747653bafcab2796`, throwaway runtime home directories, and generated provider configuration. The final aggregate loaded 397 benchmark result JSON files and 183 latency-gate result files.

In the primary controlled-core phase, a repeated 3-attempt latency/protocol gate admitted `qwen2.5-coder:7b`, `qwen3.5:9b`, and `ornith:9b`; it excluded `mistral:7b`, `lrm2.5:latest`, and `ibm/granite4.1:8b` before full runs. Across ten independent process repeats per admitted model and suite, `qwen2.5-coder:7b` achieved 5.00/5 on both smoke and agentic tasks with 3.3-3.5 seconds per task. `ornith:9b` scored 3.50/5 and 4.25/5 at 8.7 and 7.7 seconds per task. `qwen3.5:9b` scored 3.45/5 and 4.25/5 at 12.1 and 12.5 seconds per task; the nonzero smoke confidence interval was caused by one external weather API failure. Secondary clean-room phases show that runtime scaffolding can change admission and score profiles, and that native platform results are useful deployability diagnostics but should not be treated as pure model rankings.

Keywords: Local LLMs; agent benchmarking; tool use; ReAct; Ollama; reproducibility; diagnostic evaluation; OpenClaw; Hermes; Pi; Docker; consumer GPU.

1 Introduction

Local LLM agents promise private, inexpensive, and controllable tool-using systems. A local agent should be able to inspect files, call small APIs, recover from tool errors, and synthesize observed evidence into a final answer without relying on a hosted frontier model. Yet practitioners repeatedly encounter a more basic problem: a local model “cannot use tools.” That phrase is too broad to be scientifically useful.

When a local agent fails to list a directory or answer from a file, several layers may be responsible. The model may ignore the instruction to use a tool. It may choose the correct tool but emit malformed JSON. It may call an allowed tool with the wrong argument. It may call the tool successfully and then ignore the returned observation. The runtime may fail to expose tools, impose too much context overhead, or time out. A single aggregate score cannot distinguish these cases.

Local Agent Bench was built to make these distinctions explicit. It records both scores and failure reasons for each task, keeps latency visible, and separates controlled protocol testing from platform-native runtime behavior. This paper reports the first clean-room, repeated, paper-grade dataset that includes all three intended layers: primary controlled-core evidence, secondary controlled runtime-effect evidence, and secondary platform-native diagnostics.

The primary scientific claim remains scoped to the controlled-core ReAct protocol, because it is the fairest model-to-model comparison. The OpenClaw, Hermes, and Pi phases are now included as clean-room secondary evidence rather than future work. They answer different questions: how much CLI runtime scaffolding changes controlled ReAct behavior, and whether native platform use cases complete under each runtime’s own tool system.

This work makes five contributions:

- It defines a diagnostic benchmark structure for local LLM tool use that separates scoring from failure classification.
- It introduces a clean-room reproducibility workflow for both raw-Ollama and platform-runtime experiments.
- It reports a repeated controlled ReAct evaluation of six local models under a fixed WSL2 GPU execution profile.

- It reports clean-room OpenClaw, Hermes, and Pi secondary results with pinned runtime binaries and repeated independent runs.
- It shows that repeated first-turn admission gates can remove unusable model/runtime combinations early while preserving exclusions as auditable evidence.

2 Background

2.1 Tool-using language model agents

Tool-using LLM agents extend language models with external actions such as file reads, shell commands, API calls, search, and browser navigation. The model no longer only predicts text; it must decide when information is missing, select an operation, provide valid arguments, observe the result, and condition the next response on that observation.

This introduces new failure modes. An answer can be linguistically plausible while being unsupported by tool output. A model can reason correctly but fail a strict tool parser. A platform can expose the wrong tool set or hide the trace needed for evaluation. For local models, latency and context windows also become first-order constraints. A model that eventually answers correctly after two minutes per task may be a successful batch processor, but it is not an interactive local agent.

2.2 ReAct as a controlled protocol

ReAct combines reasoning and acting by alternating between textual reasoning, an action, an observation, and a final answer. Local Agent Bench uses a simple text ReAct protocol:

```
Thought: brief reasoning
Action: tool_name
Action Input: {"argument": "value"}
```

or:

```
Final Answer: concise answer to the user
```

The benchmark executes tool calls locally and appends their results as observations. This protocol is intentionally plain text. It works with any model that can follow text instructions and does not depend on a runtime's function-calling API. The tradeoff is that formatting precision becomes part of the task.

2.3 Diagnostic rather than only ordinal evaluation

Many benchmark reports produce a ranking without explaining the layer of failure. Local Agent Bench instead records both a score and a failure reason for every task. Failure reasons include NO_TOOL_ATTEMPT, INVALID_TOOL_SYNTAX, WRONG_TOOL, MISSING_REQUIRED_TOOL, IGNORED_TOOL_RESULT, HALLUCINATED_RESULT, TIMEOUT, and RUNTIME_ERROR.

This distinction matters operationally. A model that fails with INVALID_TOOL_SYNTAX may need a different prompt or native function-calling support. A model that fails with MISSING_REQUIRED_TOOL may understand the task but fail multi-step planning. A model that fails with TIMEOUT may be capable but unusable on the declared hardware. A runtime that fails with RUNTIME_ERROR should not be interpreted as a weak model.

3 Research Questions

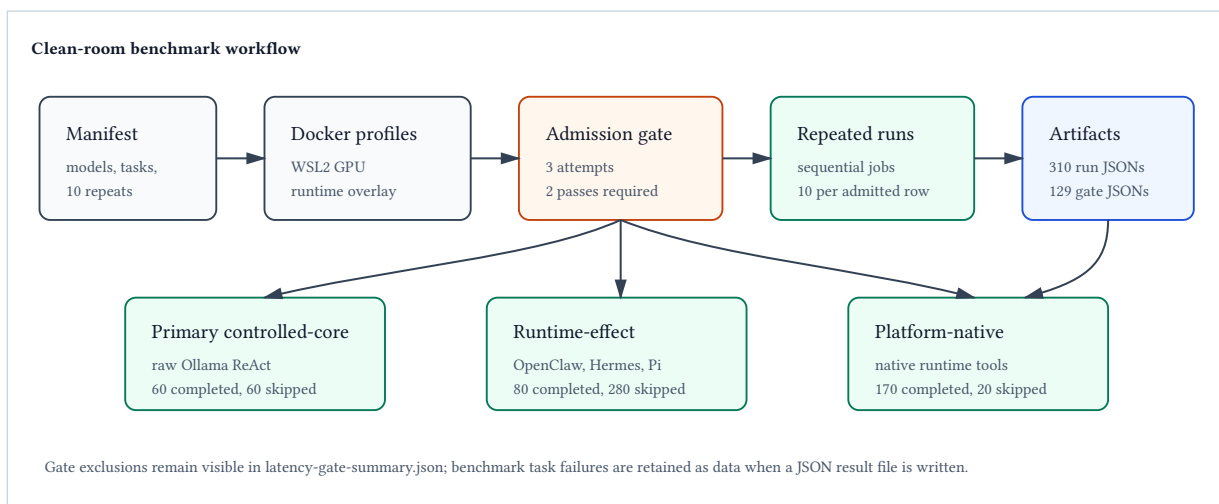


Figure 1: Clean-room workflow. The manifest defines models, tasks, and repeats; Docker fixes the execution profile; repeated latency/protocol gates admit or exclude combinations before benchmark runs are aggregated into paper artifacts.

- **RQ1.** Which local 7B-9B class models pass a repeated minimal ReAct gate under a clean WSL2 GPU profile?
- **RQ2.** Among admitted models, which complete filesystem, weather, recovery, grounding, synthesis, and tool-discipline tasks most reliably?
- **RQ3.** How do OpenClaw, Hermes, and Pi change controlled ReAct behavior when installed in the same clean-room profile?
- **RQ4.** Which runtime/model combinations complete platform-native tasks through each platform’s own tool system?
- **RQ5.** Which failure modes and validity threats explain the observed differences?

4 Methods

4.1 Benchmark layers

The benchmark harness is implemented in Python as `local_agent_bench`. It loads benchmark task files, builds a transcript, calls a backend adapter, parses either tool calls or final answers, executes benchmark-owned tools when appropriate, records observations, and scores final results. Each task produces a JSON record containing the transcript, tool calls, assertion results, latency, score, and failure reason.

The primary layer is `raw-ollama-react`. It calls the Ollama HTTP API directly, sets `temperature=0.0`, and uses the benchmark’s ReAct prompt and parser. This layer minimizes runtime-specific scaffolding and keeps the model, prompt, and local serving stack as the main variables.

The secondary controlled runtime-effect layer uses `openclaw-react`, `hermes-react`, and `pi-react`. These backends still run the benchmark-owned ReAct loop, but assistant turns are produced by each CLI runtime. The goal is to measure whether runtime prompt scaffolding and adapter behavior change controlled ReAct performance.

The platform-native layer uses `openclaw-native`, `hermes-native`, and `pi-native`. Here the runtime owns the agent turn and tool formation. Local Agent Bench scores observable evidence in the platform output and records native trace indicators when available. These results are diagnostic deployability evidence, not pure model rankings.

4.2 Tools and task suites

The controlled ReAct harness exposes four tools: `get_cwd`, `list_directory`, `read_file`, and `get_weather`. Filesystem tools are deterministic. The weather tool depends on Open-Meteo and is therefore listed as a threat to validity.

The controlled-core and runtime-effect phases use two five-task suites. The smoke suite tests current-directory reporting, project listing, fixture reading, Berlin weather lookup, and a multi-step read-then-weather task. The agentic suite adds bad-path recovery, grounding over prior knowledge, two-file synthesis, weather tool discipline, and conditional branching based on observed file content. Each run is scored out of 5.

The native-basic suite contains platform-native use cases scored out of 5. The native-ladder suite is a harder ceiling probe scored out of 7.

4.3 Models

Model	Family or role in manifest
qwen2.5-coder:7b	Qwen2.5-Coder, controlled baseline winner
qwen3.5:9b	Qwen3.5, agentic reasoning candidate
ornith:9b	Qwen3.5 fine-tune, native Pi candidate
mistral:7b	Common local baseline
lrm2.5:latest	Fast mixture-of-experts candidate
ibm/granite4.1:8b	Enterprise local baseline

Table 1: Models included in the clean-room manifest.

All models in Table 1 were pulled into the Docker-managed Ollama volume before the matrix run. Model installation was verified through the Ollama API and `ollama list`.

4.4 Clean-room execution profile

The primary repeated run used the `docker-compose.wsl-gpu` profile. This profile keeps the benchmark and Ollama inside Docker Compose while mounting WSL2 GPU resources into the Ollama container: `/dev/dxg`, `/usr/lib/wsl/lib`, and `/usr/lib/wsl/drivers`. The Ollama service logs reported CUDA compute on an NVIDIA RTX PRO 1000 Blackwell Generation Laptop GPU with 8 GB VRAM.

Runtime phases used `docker-compose.wsl-gpu.agent-runtimes`, which layers `docker-compose.runtimes.yml` onto the same Ollama service. The runtime image installs OpenClaw 2026.6.11, Pi 0.80.3, Hermes from NousResearch/hermes-agent at `f64e4f4f5768c18a53f44890747653bafcab2796`, Node.js 26.1.0, and clean `/tmp/local-agent-bench-*` runtime homes. `scripts/setup_runtime_clean_room.py` configures all runtime providers against `http://ollama:11434` or its OpenAI-compatible `/v1` endpoint.

Field	Value
Dataset	clean-room-2026-07
Primary profile	docker-compose.wsl-gpu
Runtime profile	docker-compose.wsl-gpu.agent-runtimes
Python	3.12.13
Platform	Linux 6.6.87.2 Microsoft WSL2, x86_64, glibc 2.41
Ollama version	0.31.1
Ollama image digest	ollama/ollama@sha256:f1a705f2bd113fb8d15f85f7c217f0dc5f6bebdba6b0cc42b82c3ad165ffcb9dc
OpenClaw / Pi / Hermes	2026.6.11 / 0.80.3 / f64e4f4f5768c18a53f44890747653bafcab2796

Table 2: Recorded clean-room execution metadata.

The benchmark container stamps `metadata.execution_profile` into every JSON result. The generated aggregate summary reports both execution profiles to prevent accidental mixing of raw, runtime, CPU, and GPU datasets.

4.5 Latency and protocol gate

Before full runs, the matrix runner applies a pre-registered repeated gate once per phase/runtime/model combination. The controlled ReAct gate first warms the model through the Ollama API, then runs `benchmarks/latency_gate.json` three times. The gate requires a minimal ReAct tool round-trip: call `get_cwd`, then use the observed current working directory in a final answer.

Admission requires at least two successful attempts out of three. The controlled-core threshold is 20 seconds per gate task. Runtime-effect and platform-native phases use a 120-second threshold because the objective is runtime integration diagnostics, not an interactive raw-model screen. Native full runs also use a futility gate: after three completed full runs, remaining repeats are skipped if the maximum observed native total score is at or below 1.0. Gate JSON files, `latency-gate-summary.json`, and `futility-summary.json` are retained and rendered in the generated result summary.

4.6 Repetition and aggregation

Every admitted phase/runtime/model/benchmark combination is run ten times as independent processes. Runs are sequential, not parallel, to avoid GPU contention and latency inflation. The final aggregate loaded 397 benchmark result JSON files and 183 latency-gate result JSON files.

Phase	Jobs	Completed	Skipped	Hard fails	Elapsed
controlled-core	120	60	60	0	2554.9 s
controlled-runtime-effect	360	80	280	0	4540.2 s
platform-native-basic	180	118	62	0	9544.9 s
platform-native-ladder	10	0	10	0	2117.2 s
platform-native-reality	180	138	42	0	24333.6 s

Table 3: Clean-room matrix completion counts. Skips are latency/protocol gate exclusions, not missing data.

Aggregation was performed from JSON files only. The aggregator computes mean total score, run-level standard deviation, t-based 95% confidence intervals, average latency per task, and failure reason counts. In deterministic controlled-core runs, zero confidence intervals mean repeatability over the fixed task suite, not generalization to all agent tasks.

5 Results

5.1 Primary controlled-core gate

Three of six models passed the repeated minimal ReAct gate in the primary phase. Three were excluded for protocol failure rather than raw slowness.

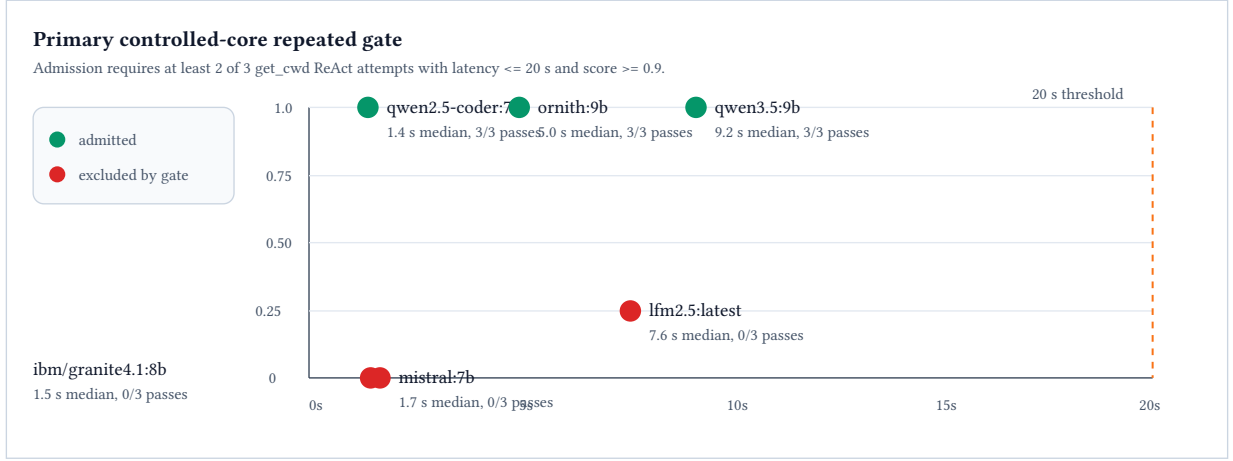


Figure 2: Primary controlled-core latency/protocol gate outcomes. Admission required at least two successful attempts out of three.

Model	Gate	Median latency	Passes	Failure reason
qwen2.5-coder:7b	pass	1.4 s	3/3	PASS
qwen3.5:9b	pass	9.2 s	3/3	PASS
ornith:9b	pass	5.0 s	3/3	PASS
mistral:7b	excluded	1.7 s	0/3	score 0.00
lfm2.5:latest	excluded	7.6 s	0/3	score 0.25
ibm/granite4.1:8b	excluded	1.5 s	0/3	score 0.00

Table 4: Pre-registered controlled-core gate results.

The gate demonstrates why protocol screening is necessary. `mistral:7b` and `ibm/granite4.1:8b` were fast, but did not complete the required tool protocol. `lfm2.5:latest` was also below the latency threshold, but failed the minimal ReAct round-trip. These models may still be useful through different prompting or native runtime interfaces, but they did not qualify for the controlled ReAct ranking.

5.2 Primary controlled-core scores

The repeated controlled-core aggregate is shown in Table 5. Each row summarizes ten independent process runs of a five-task benchmark.

Runtime	Model	Suite	Runs	Mean	95% CI	s/task	Main failures
raw-ollama-react	qwen2.5-coder:7b	smoke	10	5.00/5	+/- 0.00	3.3	PASS=50
raw-ollama-react	qwen2.5-coder:7b	agentic	10	5.00/5	+/- 0.00	3.5	PASS=50
raw-ollama-react	ornith:9b	smoke	10	3.50/5	+/- 0.00	8.7	PASS=30, WRONG_TOOL=10, MISSING_REQUIRED_TOOL=10
raw-ollama-react	ornith:9b	agentic	10	4.25/5	+/- 0.00	7.7	PASS=40, MISSING_REQUIRED_TOOL=10
raw-ollama-react	qwen3.5:9b	smoke	10	3.45/5	+/- 0.11	12.1	PASS=29, WRONG_TOOL=20, TOOL_EXECUTION_FAILED=1
raw-ollama-react	qwen3.5:9b	agentic	10	4.25/5	+/- 0.00	12.5	PASS=40, INVALID_TOOL_SYNTAX=10

Table 5: Repeated controlled-core benchmark aggregates generated from `results/paper-clean-room/summary.json`.



Figure 3: Mean score and average latency per task for primary admitted models. qwen2.5-coder:7b is both the score winner and the latency winner.

qwen2.5-coder:7b is the clear winner in the controlled setting. It is the only model to pass all tasks across both suites, and it is also the fastest admitted model. ornith:9b and qwen3.5:9b tie on agentic score. Ornith is faster, while Qwen3.5 suffered one live-weather tool failure in smoke in addition to stable wrong-tool failures.

5.3 Primary failure patterns

Model	Suite	Task group	Score	Failure pattern
qwen2.5-coder:7b	both	all smoke and agentic tasks	10.00/10 each	all pass
qwen3.5:9b	smoke	fs_list_project, fs_read_fixture	2.50/10 each	WRONG_TOOL=10 each
qwen3.5:9b	smoke	multi_step_read_then_weather	9.50/10	PASS=9, TOOL_EXECUTION_FAILED=1
qwen3.5:9b	agentic	ground_fixture_over_prior	2.50/10	INVALID_TOOL_SYNTAX=10
ornith:9b	smoke	fs_read_fixture	2.50/10	WRONG_TOOL=10
ornith:9b	smoke	multi_step_read_then_weather	2.50/10	MISSING_REQUIRED_TOOL=10
ornith:9b	agentic	recovery_bad_path_then_fixture	2.50/10	MISSING_REQUIRED_TOOL=10

Table 6: Compact per-task failure summary over ten repeated controlled-core runs.

The non-winning admitted models are not generally incapable. They fail specific protocol or planning patterns. Qwen3.5 repeatedly selects the wrong tool on two simple filesystem tasks and malformed one grounding task, while Ornith repeatedly misses required tool use on multi-step recovery and read-then-weather tasks.

5.4 Secondary controlled runtime-effect results

The runtime-effect phase ran the same controlled ReAct task suites through OpenClaw, Hermes, and Pi CLI adapters inside the runtime clean-room image. Four of eighteen runtime/model combinations passed the repeated admission gate and received ten full repeats per suite.

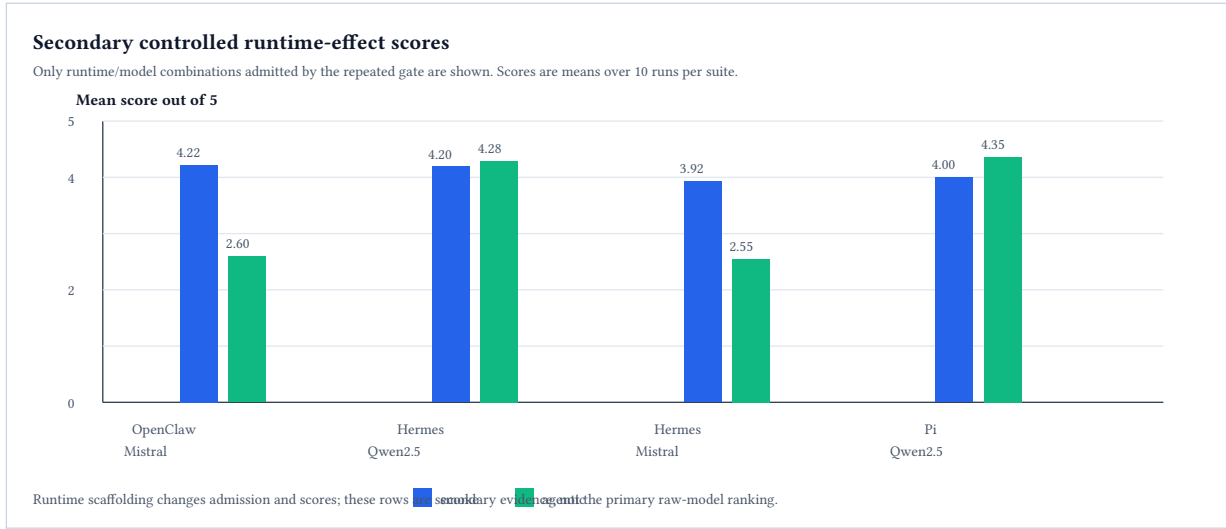


Figure 4: Secondary controlled runtime-effect results. Runtime wrappers change both admission and full-suite behavior under the same benchmark-owned ReAct tasks.

Runtime	Model	Smoke	Agentic	s/task	Main signal
openclaw-react	ollama/mistral:7b	4.22/5	2.60/5	9.1-10.8	strong smoke, weak agentic
hermes-react	ollama/qwen2.5-coder:7b	4.20/5	4.28/5	10.2-11.2	less reliable than raw
hermes-react	ollama/mistral:7b	3.92/5	2.55/5	9.6-11.0	wrapper admits model raw gate rejected
pi-react	ollama/qwen2.5-coder:7b	4.00/5	4.35/5	4.4-5.6	fastest secondary controlled result

Table 7: Admitted controlled runtime-effect combinations over ten repeats per suite.

These results are not a replacement for the primary ranking. They show that runtime prompt scaffolding and CLI adapters materially change behavior. Mistral failed the primary raw ReAct gate but passed through OpenClaw and Hermes wrappers, scoring well on smoke and poorly on agentic tasks. Qwen2.5-Coder was perfect through raw Ollama but dropped to roughly 4.0-4.35/5 through Hermes and Pi wrappers.

5.5 Secondary platform-native diagnostics

The native suites answer a different question from controlled ReAct: can a user-facing runtime complete practical work through its own tools and output surface? The use cases are current weather via native HTTP/curl and structured API data, filesystem/project inspection, concise synthesis from runtime-visible context, and a harder ladder task that combines live data with release-readiness reasoning. The expected behavior is not to emit raw tool traces. A passing answer cites or uses the right native evidence path, extracts the needed fields, handles units and location, and gives the user a short answer or decision.

Evaluation is judged from the observable runtime output. Deterministic checks are used for cheap latency gates and narrow guardrails; full native-basic and native-reality runs are scored by a local clean-room judge (ollama/mistral:7b) with explicit instructions that general web search, HTML scraping, raw JSON dumps, page placeholders, and keyword lists are failures when the task asks for structured API/native HTTP evidence. ollama/glm-5.2:cloud was used only as reference-behavior calibration because the clean Docker Ollama service could not authenticate to the cloud model.

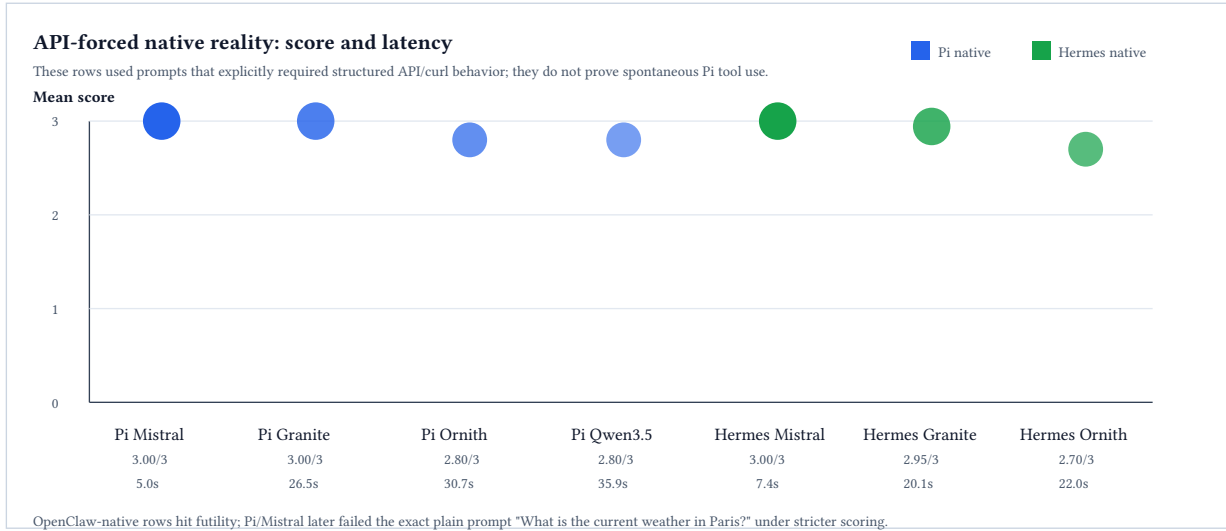


Figure 5: Platform-native reality results. Pi-native Mistral and Granite both reached 3.00/3, but Mistral did so at much lower latency.

Runtime	Suite	Rows	Score range	Latency range	Signal
openclaw-native	basic/reality	futility	0.00-0.33/3	17.7-33.2	not usable in this clean native setup
hermes-native	basic	5 full	2.67-4.90/5	9.8-28.7	Mistral/Granite strongest
hermes-native	reality	6 full	1.00-3.00/3	7.4-42.9	Mistral 3.00/3 fastest; Granite 2.95/3
pi-native	basic	5 full	4.40-5.00/5	7.5-31.9	Mistral 5.00/5 fastest
pi-native	reality	6 full	1.85-3.00/3	5.0-35.9	Mistral and Granite 3.00/3; Mistral much faster

Table 8: Platform-native aggregate summary.

The native benchmark target has been corrected: prompts should be candid user requests, while evaluator-side checks verify whether the runtime solved the use case. The previous Pi-native Mistral result should be interpreted as answer quality under over-specified prompts, not proof of spontaneous native tool use. Pi --print exposes no structured tool trace, and later inspection showed `tool_calls: []`. A targeted regression using the exact plain user prompt, “What is the current weather in Paris?”, caused Pi/Mistral to write web-search or curl instructions instead of executing a tool; under the stricter verifier it scored 0.0 on that task. Pi-native Granite has the same caveat and had already failed Emman’s manual vague-weather session.

A manual Pi session with Granite on “what is the current weather in Paris?” exposed a real failure mode in which the model returned unsynthesized search/page content and unrelated keyword summaries. A follow-up Pi/Mistral session exposed the analogous failure: it described how to use `web_search`, `web_fetch`, `curl`, or an API key, but did not actually call a tool. The corrected conclusion is that no Pi-native model is yet proven reliable for spontaneous plain-language weather tool use. These failures are not just scores; they are diagnostic artifacts that can be turned into upstream issues for the relevant runtime/model integrations.

6 Discussion

6.1 Qwen2.5-Coder is the controlled ReAct winner

The strongest primary finding is that `qwen2.5-coder:7b` outperformed the larger 9B candidates on both score and latency in the raw controlled ReAct protocol. It passed every task in both suites across all ten repeats, including multi-step tool chaining, error recovery, grounding, and tool discipline. Within this task suite, format compliance and tool-use behavior mattered more than nominal model size.

This finding should not be generalized to all agentic workloads. The benchmark tasks are small and controlled. They do not require long-horizon planning, browser control, code editing, or large-context synthesis. The correct claim is narrower: under this clean-room ReAct protocol, Qwen2.5-Coder was the most deployable controlled model.

6.2 Runtime scaffolding changes behavior

The runtime-effect phase confirms that a model is not the only variable. Mistral failed the raw controlled gate, yet OpenClaw and Hermes wrappers admitted it and achieved useful smoke scores. Conversely, Qwen2.5-Coder was perfect through raw Ollama but less reliable through Hermes and Pi CLI adapters. Runtime scaffolding can rescue protocol compliance, introduce new failure modes, or both.

This justifies keeping controlled-core and runtime-effect results separate. The first estimates model behavior under a benchmark-owned protocol. The second estimates how a model behaves after a real agent runtime has wrapped it in prompts, context, flags, and output conventions.

6.3 Native phases are deployability diagnostics

Native platform results are meaningful because they were rerun in the clean-room runtime image with repeated independent runs and recorded binaries. They should still be interpreted differently from the primary controlled benchmark. In native mode, each platform owns tool registration, context injection, permissions, and trace visibility. A 4.00/5 native score may reflect runtime constraints as much as model capability. Final-answer quality matters: returning raw search results, page placeholders, raw JSON, or keyword lists is a task failure when the requested behavior is a concise answer from a structured API.

The clean-room native results are therefore best read as deployability diagnostics. OpenClaw-native hit the futility gate in this setup and should not be treated as usable native evidence. Hermes-native was strong under the earlier over-specified prompts. Pi-native remains unresolved for spontaneous tool use: the plain weather regression shows Mistral can plan tool use instead of doing it.

6.4 Repeated gates saved compute without hiding exclusions

The repeated gate was useful. It excluded combinations that were fast but protocol-incompatible, and it avoided spending full ten-run matrices on combinations that could not reliably complete a minimal tool exchange. The 2-of-3 rule also reduced one-shot stochasticity: a single lucky or unlucky gate attempt did not decide admission.

The exclusion policy is visible rather than silent. Gate JSON files are retained, and the generated summary lists excluded combinations and reasons. This makes the ranking conditional on admission without pretending excluded combinations were never evaluated.

7 Threats to Validity

The primary dataset contains two five-task suites. Ten repeats provide run-level stability estimates, but the task corpus is small. Broader claims require more task diversity, including more filesystem tasks, longer chains, web grounding, browser interaction, code modification, and project-level workflows.

The controlled runtime uses `temperature=0.0`. This improves reproducibility and makes failures easier to diagnose, but repeated runs may be identical. Zero confidence intervals in controlled-core rows are evidence of repeatability over the fixed task suite, not evidence that broader performance is known exactly.

The final run used one WSL2 GPU profile on one 8 GB NVIDIA laptop GPU. Latency findings are hardware-specific. Score findings may also shift if model quantization, context settings, Ollama versions, runtime versions, or GPU memory pressure change.

Some tasks call a live weather API. One qwen3.5 smoke repeat encountered an Open-Meteo HTTP 503 during the multi-step weather task. The harness recorded it as `TOOL_EXECUTION_FAILED`, which is correct evidence for this run, but it also shows that live external dependencies can add non-model variance.

The latency/protocol gate excludes models that fail a minimal tool round-trip. This is appropriate for the controlled ReAct ranking, but it means excluded models do not receive full smoke/agent scores in that phase. The correct interpretation is “not admitted to this protocol comparison,” not “incapable under every possible runtime.”

Native runtime scoring is weaker than controlled ReAct scoring because native traces are not equally structured across platforms. Pi --print output in particular is less transparent than a benchmark-owned ReAct transcript. The judged native-reality suite is intended to measure user-facing answer quality, but it still depends on an LLM judge. GLM 5.2 was not used as the clean-room judge because the Docker Ollama service could not authenticate to the cloud model; the run used local Mistral 7B as judge and kept GLM 5.2 only as a reference-behavior calibration. Native scores should therefore be treated as usability diagnostics.

8 Reproducibility

The clean-room workflow is implemented in the repository and should be rerun from the manifest rather than from ad hoc shell commands. Primary files are `paper/clean-room-matrix.json`, `paper/methodology-clean-room.md`, `benchmarks/latency_gate.json`, `benchmarks/platform_native_latency_gate.json`, `benchmarks/smoke.json`, `benchmarks/agent.json`, `benchmarks/platform_native.json`, `benchmarks/platform_native_ladder.json`, `scripts/run_paper_matrix.py`, `scripts/setup_runtime_clean_room.py`, `scripts/aggregate_paper_results.py`, `docker-compose.yml`, `docker-compose.wsl-gpu.yml`, and `docker-compose.runtimes.yml`.

Primary commands:

```
docker compose -f docker-compose.yml -f docker-compose.wsl-gpu.yml up -d ollama
```

```

docker compose -f docker-compose.yml -f docker-compose.wsl-gpu.yml run --rm bench \
python3 scripts/run_paper_matrix.py --phase controlled-core --with-latency-gate --force

docker compose -f docker-compose.yml -f docker-compose.wsl-gpu.yml -f docker-compose.runtimes.yml run --rm
bench \
python3 scripts/run_paper_matrix.py --phase controlled-runtime-effect --with-latency-gate --force

docker compose -f docker-compose.yml -f docker-compose.wsl-gpu.yml -f docker-compose.runtimes.yml run --rm
bench \
python3 scripts/run_paper_matrix.py --phase platform-native-basic --with-latency-gate --force

docker compose -f docker-compose.yml -f docker-compose.wsl-gpu.yml -f docker-compose.runtimes.yml run --rm
bench \
python3 scripts/run_paper_matrix.py --phase platform-native-ladder --with-latency-gate --force

docker compose -f docker-compose.yml -f docker-compose.wsl-gpu.yml -f docker-compose.runtimes.yml run --rm
bench \
python3 scripts/run_paper_matrix.py --phase platform-native-reality --with-latency-gate --force

python3 scripts/aggregate_paper_results.py

```

Generated artifacts are `results/paper-clean-room/summary.json`, `results/paper-clean-room/latency-gate-summary.json`, and `paper/clean-room-results.md`. Tables in this paper were checked against those generated artifacts.

9 Conclusion

This paper reported a clean-room repeated evaluation of local LLM tool use across controlled raw-Ollama, controlled runtime-effect, and platform-native layers. The primary result is that `qwen2.5-coder:7b` was the only model to pass every controlled smoke and agentic task across ten independent repeats, while also being the fastest admitted controlled model. `ornith:9b` and `qwen3.5:9b` were viable but less accurate and slower. `mistral:7b`, `lrm2.5:latest`, and `ibm/granite4.1:8b` were excluded from the primary controlled ranking by the repeated ReAct gate.

The secondary result is that clean-room runtime integration matters. OpenClaw, Hermes, and Pi changed admission and score profiles in controlled ReAct mode, and their native modes produced useful deployability diagnostics that should be interpreted separately from pure model rankings. The native Pi result is now explicitly unresolved for spontaneous tool use: the exact plain weather prompt shows `mistral:7b` may plan tool use instead of doing it. The broader lesson is methodological: local agent benchmarking should identify whether a failure is protocol formatting, tool selection, missing required action, ignored observation, timeout, runtime error, or external dependency, and it should state clearly which layer produced each claim.

Future work should expand the task corpus, add task-level resampling, replace live API dependencies with replayable fixtures where possible, improve structured native traces, and compare text ReAct with native function calling. The current dataset is not the final word on local agents, but it is a reproducible foundation for studying where local tool use fails and which local model/runtime combinations are ready for practical workflows.

10 References

1. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. International Conference on Learning Representations, 2023.
2. Liang, P. et al. Holistic Evaluation of Language Models. Transactions on Machine Learning Research, 2023.
3. Liu, X. et al. AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688, 2023.
4. Schick, T. et al. Toolformer: Language Models Can Teach Themselves to Use Tools. Advances in Neural Information Processing Systems, 2023.
5. Qin, Y. et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. International Conference on Learning Representations, 2024.
6. Jimenez, C. E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? International Conference on Learning Representations, 2024.